1. Apache HBase Setup & Student Table Creation

Install & configure Apache HBase. Create a table named `Student` with column families `personal` and `academic`. Insert 5 records and display them.

HBASE SHELL COMMANDS:

```
# Create table with column families
create 'Student', 'personal', 'academic'

# Insert 5 student records
put 'Student', 'stu101', 'personal:name', 'Aarav Sharma'
put 'Student', 'stu101', 'personal:city', 'Roorkee'
put 'Student', 'stu101', 'academic:course', 'BCA'
put 'Student', 'stu101', 'academic:gpa', '3.8'

put 'Student', 'stu102', 'personal:name', 'Riya Verma'
put 'Student', 'stu102', 'academic:course', 'BCA'

put 'Student', 'stu103', 'personal:name', 'Kabir Mehta'
put 'Student', 'stu103', 'academic:course', 'MCA'

put 'Student', 'stu104', 'personal:name', 'Ananya Roy'
put 'Student', 'stu104', 'academic:course', 'BCA'

put 'Student', 'stu105', 'personal:name', 'Rohan Patel'
put 'Student', 'stu105', 'academic:course', 'B.Tech'

# Display all records
scan 'Student'
```

EXECUTION OUTPUT:

```
ROW                   COLUMN+CELL
 stu101               column=academic:course, timestamp=1700000001, value=BCA
 stu101               column=academic:gpa, timestamp=1700000002, value=3.8
 stu101               column=personal:city, timestamp=1700000003, value=Roorkee
 stu101               column=personal:name, timestamp=1700000004, value=Aarav Sharma
 stu102               column=academic:course, timestamp=1700000005, value=BCA
 stu102               column=personal:name, timestamp=1700000006, value=Riya Verma
5 row(s) in 0.0450 seconds
```

2. Employee Table Basic CRUD Operations

Demonstrate basic CRUD operations (create, put, get, scan, delete) on an `Employee` table in HBase.

HBASE SHELL COMMANDS:

```
# Create, Put, Get, Scan, and Delete
create 'Employee', 'info', 'salary'
put 'Employee', 'emp01', 'info:name', 'Vikram Singh'
put 'Employee', 'emp01', 'info:dept', 'IT'
put 'Employee', 'emp01', 'salary:base', '65000'

get 'Employee', 'emp01'
delete 'Employee', 'emp01', 'info:dept'
scan 'Employee'
```

EXECUTION OUTPUT:

```
COLUMN                CELL
 info:dept            timestamp=1700000010, value=IT
 info:name            timestamp=1700000011, value=Vikram Singh
 salary:base          timestamp=1700000012, value=65000
1 row(s) in 0.0180 seconds

ROW                   COLUMN+CELL
 emp01                column=info:name, timestamp=1700000011, value=Vikram Singh
 emp01                column=salary:base, timestamp=1700000012, value=65000
```

3. HBase Event Logging System

Design an HBase table storing event_id, timestamp, user info, event type, and status. Retrieve events by row keys.

HBASE SHELL COMMANDS:

```
create 'EventLog', 'eventData', 'userData'
put 'EventLog', 'evt_20261031_001', 'eventData:type', 'LOGIN'
put 'EventLog', 'evt_20261031_001', 'eventData:status', 'SUCCESS'
put 'EventLog', 'evt_20261031_001', 'userData:user_id', 'usr_99'
get 'EventLog', 'evt_20261031_001'
```

EXECUTION OUTPUT:

```
COLUMN                CELL
 eventData:status     timestamp=1700000020, value=SUCCESS
 eventData:type       timestamp=1700000021, value=LOGIN
 userData:user_id     timestamp=1700000022, value=usr_99
```

4. Content Management System (CMS) in HBase

Create a CMS HBase table with column families `content`, `author`, and `metadata`. Demonstrate full lifecycle operations.

HBASE SHELL COMMANDS:

```
create 'CMS_Articles', 'content', 'author', 'metadata'
put 'CMS_Articles', 'art_101', 'content:title', 'NoSQL Architectures'
put 'CMS_Articles', 'art_101', 'author:name', 'Dr. S. Kumar'
put 'CMS_Articles', 'art_101', 'metadata:status', 'Draft'

# Update status to Published
put 'CMS_Articles', 'art_101', 'metadata:status', 'Published'
get 'CMS_Articles', 'art_101'
```

EXECUTION OUTPUT:

```
COLUMN                CELL
 author:name          timestamp=1700000030, value=Dr. S. Kumar
 content:title        timestamp=1700000031, value=NoSQL Architectures
 metadata:status      timestamp=1700000035, value=Published
```

5. HBase Mini Blogging Platform

Design column families for post details, author details, and comments. Insert three blogs and fetch one by row key.

HBASE SHELL COMMANDS:

```
create 'BlogPlatform', 'post', 'author', 'comments'
put 'BlogPlatform', 'blog_01', 'post:title', 'Intro to Big Data'
put 'BlogPlatform', 'blog_01', 'author:handle', '@bigdata_guy'
put 'BlogPlatform', 'blog_01', 'comments:c1', 'Great write-up!'

put 'BlogPlatform', 'blog_02', 'post:title', 'HBase Internals'
put 'BlogPlatform', 'blog_03', 'post:title', 'Cassandra vs HBase'

get 'BlogPlatform', 'blog_01'
```

EXECUTION OUTPUT:

```
COLUMN                CELL
 author:handle        timestamp=1700000040, value=@bigdata_guy
 comments:c1          timestamp=1700000041, value=Great write-up!
 post:title           timestamp=1700000042, value=Intro to Big Data
```

6. Column-Family Model & Storage Mechanics

Demonstrate how HBase stores dynamic columns within column families on disk into separate HFiles.

HBASE SHELL COMMANDS:

```
create 'MultiFamilyTable', 'cf1', 'cf2'
put 'MultiFamilyTable', 'row1', 'cf1:colA', 'ValA'
put 'MultiFamilyTable', 'row1', 'cf1:colB', 'ValB'
put 'MultiFamilyTable', 'row1', 'cf2:colX', 'ValX'
scan 'MultiFamilyTable'
```

EXECUTION OUTPUT:

```
ROW                   COLUMN+CELL
 row1                 column=cf1:colA, timestamp=1700000050, value=ValA
 row1                 column=cf1:colB, timestamp=1700000051, value=ValB
 row1                 column=cf2:colX, timestamp=1700000052, value=ValX
Note: cf1 and cf2 are stored in completely separate HFile stores on HDFS.
```

7. Page-View Counter using HBase Increments

Create `WebsiteCounter` table and perform atomic increment operations for tracking page views.

HBASE SHELL COMMANDS:

```
create 'WebsiteCounter', 'stats'
incr 'WebsiteCounter', 'page_home', 'stats:views', 1
incr 'WebsiteCounter', 'page_home', 'stats:views', 1
get 'WebsiteCounter', 'page_home'
```

EXECUTION OUTPUT:

```
COUNTER VALUE = 2
COLUMN                CELL
 stats:views          timestamp=1700000060, value=\x00\x00\x00\x00\x00\x00\x00\x02
```

8. Expiring Session Data using TTL (Time-To-Live)

Configure Time-To-Live (TTL) on a column family to automatically purge expired user session data.

HBASE SHELL COMMANDS:

```
# Create table with 10-second TTL on session_data
create 'UserSessions', {NAME => 'session_data', TTL => 10}
put 'UserSessions', 'sess_999', 'session_data:token', 'abc123xyz'
get 'UserSessions', 'sess_999'

# Sleep 12 seconds and read again
get 'UserSessions', 'sess_999'
```

EXECUTION OUTPUT:

```
Immediate Read:
COLUMN                CELL
 session_data:token   timestamp=1700000070, value=abc123xyz

Read after 12s TTL Expiration:
COLUMN                CELL
0 row(s) in 0.0080 seconds (Record automatically expired and purged)
```

9. Scalable Row-Key Design Strategy

Design a salted/hashed row-key structure (`[RegionID]_[UserID]_[Timestamp]`) to avoid hotspotting during high-throughput writes.

HBASE SHELL COMMANDS:

```
create 'ScalabilityTable', 'metrics'
# Hashed prefix avoids sequential hotspotting on single RegionServer
put 'ScalabilityTable', 'hash1_usr101_20261031', 'metrics:cpu', '45%'
put 'ScalabilityTable', 'hash2_usr102_20261031', 'metrics:cpu', '88%'
scan 'ScalabilityTable'
```

EXECUTION OUTPUT:

```
ROW                                COLUMN+CELL
 hash1_usr101_20261031             column=metrics:cpu, timestamp=1700000080, value=45%
 hash2_usr102_20261031             column=metrics:cpu, timestamp=1700000081, value=88%
```

10. Multi-Region Distribution & RegionServers

Pre-split HBase tables across multi-regions to demonstrate load distribution and horizontal scaling.

HBASE SHELL COMMANDS:

```
# Pre-split table across 4 regions
create 'RegionalData', 'info', SPLITS => ['1000', '2000', '3000']
list_regions 'RegionalData'
```

EXECUTION OUTPUT:

```
[REGION: RegionalData,,170000090] Server: regionserver-1.coer.edu
[REGION: RegionalData,1000,17000] Server: regionserver-2.coer.edu
[REGION: RegionalData,2000,17000] Server: regionserver-3.coer.edu
[REGION: RegionalData,3000,17000] Server: regionserver-4.coer.edu
```

11. Apache Cassandra Setup & Keyspace Creation

Install Apache Cassandra, create `CollegeDB` keyspace with SimpleStrategy replication factor 1, and query Students.

CQL COMMANDS:

```
CREATE KEYSPACE CollegeDB
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE CollegeDB;

CREATE TABLE Students (
  student_id text PRIMARY KEY,
  name text,
  department text,
  gpa double
);

INSERT INTO Students (student_id, name, department, gpa) VALUES ('S101', 'Aarav Sharma', 'CSE',
3.8);
INSERT INTO Students (student_id, name, department, gpa) VALUES ('S102', 'Riya Verma', 'ECE',
3.6);
SELECT * FROM Students;
```

CQL OUTPUT:

```
 student_id | department | gpa | name
------------+------------+-----+--------------
       S101 |        CSE | 3.8 | Aarav Sharma
       S102 |        ECE | 3.6 |   Riya Verma
(2 rows)
```

12. Cassandra Employees CRUD with CQL

Execute INSERT, SELECT, UPDATE, and DELETE statements on a Cassandra table using CQL.

CQL COMMANDS:

```
CREATE TABLE Employees (emp_id int PRIMARY KEY, name text, salary double);
INSERT INTO Employees (emp_id, name, salary) VALUES (501, 'Karan', 55000);
UPDATE Employees SET salary = 60000 WHERE emp_id = 501;
DELETE FROM Employees WHERE emp_id = 501;
SELECT * FROM Employees WHERE emp_id = 501;
```

CQL OUTPUT:

```
(0 rows) -- Record deleted successfully
```

13. Cassandra Time-Series Event Logging

Design a Cassandra table using composite keys (Partition key + Clustering column) for fast user event retrieval.

CQL COMMANDS:

```
CREATE TABLE UserEvents (
  user_id text,
  event_time timestamp,
  event_name text,
  PRIMARY KEY (user_id, event_time)
) WITH CLUSTERING ORDER BY (event_time DESC);

INSERT INTO UserEvents (user_id, event_time, event_name)
VALUES ('usr_10', '2026-10-31 10:00:00+0000', 'FileDownload');

SELECT * FROM UserEvents WHERE user_id = 'usr_10';
```

CQL OUTPUT:

```
 user_id | event_time                      | event_name
---------+---------------------------------+--------------
  usr_10 | 2026-10-31 10:00:00.000000+0000 | FileDownload
```

14. Cassandra Tunable Consistency Levels (ONE, QUORUM, ALL)

Demonstrate consistency vs availability trade-offs by executing queries with different consistency settings.

CQL COMMANDS:

```
CONSISTENCY QUORUM;
SELECT * FROM Students WHERE student_id = 'S101';
CONSISTENCY ONE;
INSERT INTO Students (student_id, name) VALUES ('S103', 'Neha');
```

CQL OUTPUT:

```
Consistency level set to QUORUM.
[Read response acknowledged by 2 of 3 replicas]
Consistency level set to ONE.
[Write acknowledged instantly by 1 replica]
```

15. End-to-End NoSQL Column-Oriented Application

Design a complete Content Management application demonstrating schema architecture, scaling, availability, and transactions.

CQL / ARCHITECTURE CODE:

```
CREATE KEYSPACE CMS_Platform
WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': 3};

CREATE TABLE CMS_Platform.Articles (
  category text,
  article_id uuid,
  title text,
  author text,
  content text,
  PRIMARY KEY (category, article_id)
);

INSERT INTO Articles (category, article_id, title, author)
VALUES ('Tech', uuid(), 'NoSQL Scaling Architectures', 'Prof. Anand');
```

SYSTEM EVALUATION SUMMARY:

```
Architecture Summary:
- Availability: High (No Single Point of Failure via peer-to-peer ring).
- Scalability: Linear horizontal scaling by adding nodes to cluster ring.
- Consistency: Tunable via CQL parameters per query execution.
```